

S. Задание к лабораторным работам по АиСД

Касилов Василий

28 февраля 2026 г.

Версия 1.0

Задания к лабораторным работам 2-го семестра по дисциплине «Алгоритмы и структуры данных» выполняются и принимаются по порядку. В рамках лабораторных работ не разрешается использовать стандартные контейнеры и `std::stringstream`. Однако допускается использование `std::pair`, `std::tuple`, `std::function` и для обработки ввода `std::string`, если задание явно не запрещает этого. Для реализации модульных тестов допустимо использовать любые классы стандартной библиотеки.

Все реализуемые сущности (за исключением функции `main` должны быть расположены в отдельном пространстве имен. Имя этого пространства должно совпадать с фамилией студента в нижнем регистре (соответственно, оно совпадает с частью имени каталога с работами до точки). Например, для Петрова Ивана каталог будет называться `petrov.ivan`, а имя пространства имен — `petrov`. Это пространство имен должно использоваться для всех работ. Для функций, являющихся деталью реализации допустимо использование анонимного пространства имён.

0 Вступительная работа

1. Реализуйте программу, которая выводит на стандартный вывод фамилию и имя студента, разделённые символом «.», на отдельной строке
2. Работа должна быть выполнена в виде 1-го исполняемого файла, не обрабатывающего параметры командной строки:

```
$ ./lab
petrov.ivan
```

1 Списки I

Реализуйте программу, которая считывает последовательности чисел со стандартного ввода и обрабатывает их в соответствии с заданием

1. Входные данные содержат именованные последовательности чисел — по одной на каждой строке. В следующем примере введено четыре таких последовательности:

```
first 1 1 1
second 2 2 2 2
third
fourth 4 4
```

Количество чисел в последовательностях может различаться. Все числа целые, положительные — проверка на корректность членов последовательности не требуется. Гарантируется, что название состоит только из букв латинского алфавита и ровно одного слова. Признаком конца ввода является EOF (на Linux: `Ctrl + D` | на Windows `Ctrl + Z` затем `Enter`)

2. В работе допустимо использовать класс строки из стандартной библиотеки только для хранения названий. Числа должны быть разобраны с помощью встроенного оператора ввода: преобразование из строки не допускается.
3. Программа должна вывести следующие результаты:
 - Список названий введённых списков на отдельной строке
 - Затем со следующей строки последовательности чисел, составленные из элементов введённых по следующему правилу: сначала идёт последовательность из первых чисел всех введённых последовательностей (первое число первой последовательности, первое число второй последовательности и т.д.), затем последовательность вторых чисел и так далее, пока не закончатся числа во всех последовательностях. Числа каждого списка включаются в результат ровно один раз. Каждая последовательность выводится на отдельной строке
 - Затем на следующей строке список сумм значений составленных списков на отдельной строке, если такой расчёт возможен

Например, для указанных входных данных в качестве результата ожидается следующий вывод:

```
first second third fourth
1 2 4
1 2 4
```

```
1 2
2
7 7 3 2
```

4. Если ввод не содержит последовательностей, программа должна вывести 0
5. Если расчёт требуемой суммы невозможен, программа должна завершаться с соответствующим сообщением об ошибке и кодом возврата 1. Во всех остальных случаях программа завершается с кодом возврата 0
6. Для хранения чисел необходимо реализовать шаблон класса списка и соответствующие итераторы для константного и неконстантного обходов

```
1  template< class T > class List;
2  template< class T >
3  class LIter {
4      friend class List< T >;
5      ...
6  };
7  template< class T >
8  class LCIter {
9      friend class List< T >;
10     ...
11 };
12 template< class T >
13 class List {
14     ...
15 };
```

Разрабатываемый интерфейс должен быть безопасным относительно исключений

7. Интерфейс для однонаправленного и двунаправленного списка может различаться. Например, не следует реализовывать добавление элементов в конце однонаправленного списка в виде отдельного метода, так как это провоцирует пользователей писать неэффективный код. Но должна быть возможность реализовать ту же логику через метод, добавляющий элементы после позиции, указанной итератором (см. метод `insert` для стандартных списков).
8. Уместно поддерживать следующие операции:
 - (a) Получение итераторов начала и конца
 - (b) Доступ к элементам «с краю» списка
 - (c) Добавление элементов в начало и конце списка
 - (d) Удаление элементов из начала и конца списка
 - (e) Очищение списка
 - (f) Копирование и перемещение

Совет-1 При реализации методов списка следует ориентироваться на контейнеры стандартной библиотеки `std::list` или `std::forward_list`

Совет-2 Сохранять введённые последовательности можно в виде списка пар (`std::pair`), где первый элемент пары — название списка, а второй — список соответствующих чисел